Práctica guiada — Inferencia y contratos

Este notebook es para el alumnado. Completa las celdas marcadas con `TODO` y responde las preguntas antes de mirar la solución docente.

Objetivo: pasar de una fila CSV a una especificación clara para el módulo de inferencia de la clase 2.

Cómo trabajar

1. Copia esta carpeta como tu espacio de trabajo.
2. Ejecuta `uv sync` desde `problem/starter/` cuando empiece la clase 2.
3. En esta práctica no implementes todavía el CLI completo.
4. Trabaja en parejas y cambia quién escribe cada diez minutos.
5. No abras `solutions/` hasta el debrief.

```
from dataclasses import dataclass
import csv
from pathlib import Path

from pydantic import BaseModel, ConfigDict, Field, ValidationError

DATA_PATH =
Path("../../../../../../assets/03-wine-quality/inference_samples.csv")
print(DATA_PATH)
print(DATA_PATH.exists())
```

```
..\..\..\..\..\..\assets\03-wine-quality\inference_samples.csv
True
```

1. Leer los datos nuevos

Práctica A — Inspección del CSV

Completa la celda siguiente para:

- Leer el CSV con `csv.DictReader`.
- Imprimir el número de filas.
- Imprimir las columnas.
- Guardar la primera fila en una variable llamada `row`.

```
with DATA_PATH.open(newline="", encoding="utf-8") as source:
    reader = csv.DictReader(source)
    rows = list(reader)

print(len(rows))
print(reader.fieldnames)
```

```
row = rows[0]
row
```

```
5
['sample_id', 'fixed_acidity', 'volatile_acidity', 'citric_acid',
'residual_sugar', 'chlorides', 'free_sulfur_dioxide',
'total_sulfur_dioxide', 'density', 'ph', 'sulphates', 'alcohol']
```

```
{'sample_id': 'red-001',
 'fixed_acidity': '7.4',
 'volatile_acidity': '0.7',
 'citric_acid': '0.0',
 'residual_sugar': '1.9',
 'chlorides': '0.076',
 'free_sulfur_dioxide': '11.0',
 'total_sulfur_dioxide': '34.0',
 'density': '0.9978',
 'ph': '3.51',
 'sulphates': '0.56',
 'alcohol': '9.4'}
```

Preguntas

1. ¿Qué campo identifica la muestra pero no es una característica del modelo?
2. ¿Cuántas características numéricas hay?
3. ¿Qué columna extra añadirías para probar el contrato?
4. ¿Qué valor de `ph` sería claramente inválido?

Respuestas

1. `sample_id`. Identifica la solicitud y viaja con la respuesta, pero no entra en el vector del modelo.
2. Hay 11 características numéricas: todas las columnas menos `sample_id`.
3. Una inesperada (`unexpected_field`). Cualquier columna fuera del contrato debe rechazarse.
4. `ph = 99`, ya que la escala química no pasa de 14. También sería inválido un valor negativo o un texto como `"abc"`.

2. Construir el contrato con Pydantic

El contrato del modelo contiene estas características:

Campo	Rango orientativo
`fixed_acidity`	0–20
`volatile_acidity`	0–2
`citric_acid`	0–2
`residual_sugar`	0–20
`chlorides`	0–1

Campo	Rango orientativo
`free_sulfur_dioxide`	0–100
`total_sulfur_dioxide`	0–300
`density`	0.98–1.01
`ph`	2.5–4.5
`sulphates`	0–3
`alcohol`	5–20

```
class WineQualityRequestPydantic(BaseModel):
    model_config = ConfigDict(extra="forbid")

    fixed_acidity: float = Field(ge=0, le=20)
    volatile_acidity: float = Field(ge=0, le=2)
    citric_acid: float = Field(ge=0, le=2)
    residual_sugar: float = Field(ge=0, le=20)
    chlorides: float = Field(ge=0, le=1)
    free_sulfur_dioxide: float = Field(ge=0, le=100)
    total_sulfur_dioxide: float = Field(ge=0, le=300)
    density: float = Field(ge=0.98, le=1.01)
    ph: float = Field(ge=2.5, le=4.5)
    sulphates: float = Field(ge=0, le=3)
    alcohol: float = Field(ge=5, le=20)

model_row = {key: value for key, value in row.items() if key !=
"sample_id"}
request = WineQualityRequestPydantic.model_validate(model_row)
request
```

```
WineQualityRequestPydantic(fixed_acidity=7.4, volatile_acidity=0.7,
citric_acid=0.0, residual_sugar=1.9, chlorides=0.076,
free_sulfur_dioxide=11.0, total_sulfur_dioxide=34.0, density=0.9978,
ph=3.51, sulphates=0.56, alcohol=9.4)
```

Práctica B — Validación

Completa una prueba para cada caso:

- Una fila válida.
- Una columna `unexpected_field`.
- Un `ph` fuera de rango.

Antes de ejecutar, escribe qué error esperas.

```
# Fila válida: no se espera error.
try:
    print(WineQualityRequestPydantic.model_validate(model_row))
except ValidationError as error:
    print(error)
```

```
# Columna extra: se espera un error debido a unexpected_field.
try:
    WineQualityRequestPydantic.model_validate(
        {**model_row, "unexpected_field": "not allowed"}
    )
except ValidationError as error:
    print(error)

# ph fuera de rango: se espera que indique el rango máximo permitido
de ph.
try:
    WineQualityRequestPydantic.model_validate({**model_row, "ph":
"99"})
except ValidationError as error:
    print(error)
```

```
fixed_acidity=7.4 volatile_acidity=0.7 citric_acid=0.0
residual_sugar=1.9 chlorides=0.076 free_sulfur_dioxide=11.0
total_sulfur_dioxide=34.0 density=0.9978 ph=3.51 sulphates=0.56
alcohol=9.4
1 validation error for WineQualityRequestPydantic
unexpected_field
  Extra inputs are not permitted [type=extra_forbidden,
input_value='not allowed', input_type=str]
    For further information visit
https://errors.pydantic.dev/2.14/v/extra_forbidden
1 validation error for WineQualityRequestPydantic
ph
  Input should be less than or equal to 4.5 [type=less_than_equal,
input_value='99', input_type=str]
    For further information visit
https://errors.pydantic.dev/2.14/v/less_than_equal
```

3. Comparar con `dataclass`

Construye la misma estructura con `dataclass`. No añadas todavía validación manual: observa qué garantiza Python y qué no garantiza.

```
@dataclass(frozen=True, slots=True)
class WineQualityRequestDataclass:
    fixed_acidity: float
    volatile_acidity: float
    citric_acid: float
    residual_sugar: float
    chlorides: float
    free_sulfur_dioxide: float
    total_sulfur_dioxide: float
    density: float
```

```
    ph: float
    sulphates: float
    alcohol: float

float_row = {key: float(value) for key, value in model_row.items()}
valid = WineQualityRequestDataclass(**float_row)
print(valid)

# ph=99.0 se acepta sin error: dataclass no valida rangos.
invalid = WineQualityRequestDataclass(**{**float_row, "ph": 99.0})
print(invalid)
```

```
WineQualityRequestDataclass(fixed_acidity=7.4, volatile_acidity=0.7,
citric_acid=0.0, residual_sugar=1.9, chlorides=0.076,
free_sulfur_dioxide=11.0, total_sulfur_dioxide=34.0, density=0.9978,
ph=3.51, sulphates=0.56, alcohol=9.4)
WineQualityRequestDataclass(fixed_acidity=7.4, volatile_acidity=0.7,
citric_acid=0.0, residual_sugar=1.9, chlorides=0.076,
free_sulfur_dioxide=11.0, total_sulfur_dioxide=34.0, density=0.9978,
ph=99.0, sulphates=0.56, alcohol=9.4)
```

Decisión de diseño

Completa la frase:

> Usaría Pydantic en _____ porque _____. Usaría `dataclass` en _____ porque _____.
>
> Usaría **Pydantic** en **la frontera del sistema (la entrada del CSV, una API o la salida que consume otro equipo)** porque **los datos vienen de fuera, así que se deben validar tipos, convertir texto a número, aplicar rangos y rechazar campos extra con un mensaje de error claro**. Usaría **`dataclass`** en **estructuras internas que se construyen con datos ya validados** porque **es ligera, inmutable con `frozen=True` y no añade dependencias, pero no comprueba ni tipos ni rangos**.

4. Definir el orden del vector

El modelo no recibe un diccionario. Recibe una lista numérica con un orden estable.

```
FEATURE_NAMES = (
    "fixed_acidity",
    "volatile_acidity",
    "citric_acid",
    "residual_sugar",
    "chlorides",
    "free_sulfur_dioxide",
    "total_sulfur_dioxide",
    "density",
    "ph",
    "sulphates",
```

```
    "alcohol",
)

vector = [getattr(request, name) for name in FEATURE_NAMES]
vector
```

```
[7.4, 0.7, 0.0, 1.9, 0.076, 11.0, 34.0, 0.9978, 3.51, 0.56, 9.4]
```

Práctica C — Error de orden

Intercambia mentalmente `density` y `alcohol`. Responde:

- ¿El vector sigue teniendo 11 valores?
- ¿El clasificador detectaría necesariamente el error?
- ¿Cómo evitarías que cada script definiera un orden distinto?

Respuestas

- **Sí.** El vector sigue teniendo 11 valores de tipo `float`, así que cualquier comprobación de forma o de tipo lo da por bueno.
- **No.** El clasificador no sabe qué significa cada posición: recibe `alcohol=0.9978` y `density=9.4` y predice igualmente. Con el modelo real, `red-003` pasa de `acceptable` (0.62) a `needs_review` (0.50) sin ningún error, y el resto de muestras cambia su confianza. Es un error silencioso.
- **Definiendo `FEATURE_NAMES` una sola vez** en el módulo de preprocesado, y construyendo el vector siempre con ese orden y no con el orden de las columnas del CSV. Además, **al cargar el artefacto se comprueba** que su `feature_names` coincide exactamente con `FEATURE_NAMES`, y si no coincide se rechaza antes de llamar a `predict`.

5. Contrato de salida

Diseña la estructura que devolverá el módulo. Debe permitir responder:

- ¿Qué muestra se ha procesado?
- ¿Qué categoría se ha predicho?
- ¿Con qué confianza?
- ¿Qué versión de modelo y preprocesado se utilizaron?

```
class WineQualityPredictionPydantic(BaseModel):
    model_config = ConfigDict(extra="forbid")

    sample_id: str = Field(min_length=1)
    quality_band: str = Field(min_length=1)
    confidence: float = Field(ge=0, le=1)
    model_version: str = Field(min_length=1)
    preprocessing_version: str = Field(min_length=1)

WineQualityPredictionPydantic(
```

```
    sample_id="red-001",
    quality_band="needs_review",
    confidence=0.86,
    model_version="wine-quality-rf-v1",
    preprocessing_version="wine-red-features-v1",
)
```

```
WineQualityPredictionPydantic(sample_id='red-001',
quality_band='needs_review', confidence=0.86, model_version='wine-
quality-rf-v1', preprocessing_version='wine-red-features-v1')
```

6. Demo del docente: observar la inferencia

El docente ejecutará el notebook resuelto y el comando de referencia. Mientras tanto, anota el recorrido:

```
CSV → contrato → preprocesado → modelo cargado → predicción
```

No copies la implementación. Tu código de producción empieza en la clase 2 con el starter.

Entrega

Entrega una captura o exportación con:

- La tabla de contrato.
- Las respuestas de las prácticas A, B y C.
- El orden de las 11 características.
- Un ejemplo de entrada inválida.
- Un contrato de salida.
- Tu plan de implementación para `contracts.py`, `preprocess.py`, `inference.py` y `predict_file.py`.